TECHNICAL ASSIGNMENT

AI Engineering & Prototyping

Agentic HCM Workflow Engine

Darwinbox — AI Engineering & Prototyping Team

PART 1
STANDARD ASSIGNMENT

This assignment evaluates three capabilities central to the role: agentic system design, retrieval-augmented generation, and production-grade thinking. It is scoped to be completable in a focused weekend.

1.1 Problem Statement

Darwinbox processes millions of HR transactions daily — leave requests, payroll queries, policy lookups, and onboarding tasks. Your goal is to build a multi-agent workflow engine that can autonomously handle a subset of these HR operations via a conversational interface.

1.2 What to Build

Core Agent Loop

- An orchestrator agent that receives a natural language HR request (e.g., Apply for 3 days leave starting June 15 or What is our maternity leave policy?)
- Route to at least 2 specialized sub-agents: one for policy lookup via RAG, one for action execution via mock API calls
- State must persist across a multi-turn conversation — the agent should remember context from earlier in the session

RAG Component

- Embed a small HR policy document (2–3 pages; you may generate a realistic mock)
- The policy agent must retrieve and ground its answers — hallucinated policy details are an automatic failure signal
- Chunking strategy, embedding model choice, and retrieval quality are all evaluated

Tool Use

- At least one mock tool call with structured JSON I/O: leave balance check, leave application, or payslip fetch
- Tool errors must be handled gracefully — retry logic or meaningful fallback responses required
- Tool schemas should be clearly defined (OpenAI-style or similar)

Observability

- A trace/log showing which agent ran, what tool was called, inputs, outputs, and latency per step
- LLM cost tracking per run — show a cost optimisation strategy reducing token spend by ≥20% vs naive all-LLM baseline (with numbers)
- Bonus: A minimal UI (Streamlit or React) showing the agent reasoning trace live

PART 2
ADVANCED ASSIGNMENT

This is a single advanced track designed for candidates who think like systems architects and research engineers. Beyond agentic systems and RAG, it requires you to design a reinforcement learning feedback layer — reasoning about learning signals, reward design, and the feedback loop between human judgment and model behavior. If this scope is outside your current experience, Part 1 is the appropriate starting point.

2.1 Problem Statement

Darwinbox serves 3M+ employees across 750+ enterprise clients in 90+ countries. Behind every payslip, leave approval, and performance review is a dense web of HR workflows — each one prone to data drift, policy violations, and human error at scale. Today, fixing these issues requires HR Ops teams to manually triage tickets, cross-reference policy documents, and escalate to payroll or compliance teams. At 3M employees, that is not sustainable.

You are being asked to build the core intelligence layer of what Darwinbox calls the —

Self-Healing HR Ops Platform

An agentic system that does not just respond to HR queries, but proactively monitors workforce data, detects anomalies before they become incidents, initiates correction workflows autonomously, and — critically — learns from every human decision to improve its future proposals.

The system handles three trigger classes:

- Reactive: An employee or HR manager submits a natural-language request (e.g. "Why was my payslip short by AED 800 this month?" or "Flag anyone in Engineering who has taken more than 15 days leave in Q1").
- Scheduled: A cron-like job runs a data scan over the mock employee dataset every cycle, proactively surfacing anomalies without any human trigger.
- System-generated: A mock upstream API (payroll engine, attendance system) emits an alert that another agent picks up and routes for investigation.

Across all three trigger types, the system must: retrieve and ground answers in HR policy documents (RAG), execute corrective actions via tool calls with guardrails, escalate high-risk decisions to a human approver (HITL), enforce compliance rules as hard vetoes, persist episodic memory of past incidents, and — most importantly — use reinforcement learning to improve its action proposals based on accumulated human feedback.

You will generate the simulated dataset (500–1,000 employee records: attendance, payroll, leave, performance), the HR policy documents (3–5 pages PDF/Markdown), and the mock API spec. How realistic and internally consistent these artefacts are is part of the evaluation itself.

2.2 What to Build

A. Multi-Agent Architecture (LangGraph or equivalent)

- Supervisor agent triages all incoming signals and delegates to sub-agents; all state transitions must be logged to a shared graph state.
- Minimum 4 sub-agents: Policy Agent (RAG), Action Agent (tool execution), Anomaly Detection Agent (data scan + scoring), Compliance Agent (hard veto rules engine).
- No direct agent-to-agent calls. All communication flows through the shared state graph. This is a hard architectural requirement.

B. Anomaly Detection Pipeline

- Over the generated employee dataset, detect: payroll outliers (statistical deviation from peer cohort), leave abuse patterns (unusual clustering relative to policy limits), and compliance violations (e.g. missing mandatory training, overtime cap breaches).
- Each detected anomaly must carry a confidence score (0–1.0) and a recommended action (one of: auto-correct, escalate-to-manager, escalate-to-HR, flag-for-audit, no-action). Scoring method is up to the candidate — justify it.
- High-confidence anomalies (above a threshold you define) should automatically trigger the Action Agent. Low-confidence ones should queue for human review.

C. Reinforcement Learning Feedback Layer — Core Requirement

This is the section that differentiates this assignment from a standard agentic systems build. The RL layer must be implemented and must demonstrably influence the system's future behaviour. A superficial implementation (e.g. appending feedback to a prompt with no learning signal) will not pass.

- Reward signals: (1) HITL decisions: manager approves +1, rejects -1, modifies partial reward based on edit distance; (2) Outcome feedback: did an auto-corrected anomaly recur within N cycles? (3) False positive rate: did a flagged anomaly turn out to be a data error or legitimate activity?
- Algorithm choice: Implement a contextual bandit for the action-selection decision, OR a lightweight PPO/REINFORCE loop on the Supervisor agent's routing policy. You must implement one; doing both is a bonus. Justify your choice.
- What must change over time: Run at least 2 simulated feedback cycles. Show that action proposals or routing decisions measurably change after incorporating feedback. A before/after comparison is required in the Loom walkthrough.
- Persistence: The learned policy or reward model must persist to disk between runs. Restarting the server should not reset the RL state.

D. Human-in-the-Loop (HITL) — Primary Feedback Source for RL

- For actions above a defined risk threshold, the system pauses and routes to a minimal approval interface (webhook endpoint or Streamlit/React UI).
- The interface must expose: the anomaly context, the proposed action, the confidence score, and the agent's reasoning. The reviewer can approve, reject, or modify.
- All decisions — including rejection reasons — must be persisted and fed back into the RL reward pipeline. This is the primary data flywheel.
- Timeout handling: if no human response within a configurable window, fall back to a safe default action (no-action or flag-for-audit).

E. Compliance Rules Engine

- Define 10–15 HR compliance rules in a YAML or JSON file (not in prompts). Examples: maximum overtime hours per week, mandatory notice period before leave, probation period constraints, payroll correction approval tiers.
- The Compliance Agent evaluates every proposed action against this ruleset and issues a hard veto for violations, even if the Supervisor or RL policy recommends the action.
- The RL reward function must penalise actions that trigger compliance vetoes, ensuring the policy learns to avoid them.

F. Episodic Memory

- Use a vector store (Chroma DB, Qdrant, or equivalent) to store past incident resolutions with their context, action taken, outcome, and reward received.
- When a new anomaly is detected, the system must retrieve semantically similar past incidents and use them to bias its action proposal — effectively warm-starting the RL policy from memory.
- Demonstrate in the walkthrough that the same anomaly type is handled faster and with higher confidence on the second occurrence vs. the first.

G. Observability and Evaluation

- Structured trace per workflow run: agent name, input, output, tool calls, latency, token usage, RL action selected, reward received.
- Evaluation harness: 15 test cases covering happy path, edge cases, adversarial inputs, and RL-specific scenarios. Report pass/fail with reasoning.
- RL diagnostics: plot or log the cumulative reward curve and the action distribution shift across feedback cycles. Required for the Loom walkthrough.
- LLM cost tracking per run — show a cost optimisation strategy reducing token spend by ≥20% vs naive all-LLM baseline (with numbers).

SUBMISSION CHECKLIST

- GitHub repo (public or shared with hiring team) — clean commit history required
- README.md with architecture diagram (even ASCII), setup instructions, and key design decisions
- Loom walkthrough: 10 minutes maximum — demo the system, explain the hardest trade-off you made, and describe what you'd change at production scale
- Architecture brief: 1-page PDF or Markdown document